



Ingegneria del software (per ripasso)

1. Cos'è il Software e lo Sviluppo Professionale

Spesso si pensa che il software sia solo codice, ma nell'ingegneria del software esso comprende i programmi, tutta la documentazione associata e i dati di configurazione necessari per farlo funzionare.

Lo sviluppo professionale del software produce due tipi principali di prodotti:

- **Prodotti generici:** Sistemi autonomi venduti sul mercato aperto a chiunque voglia acquistarli (es. database, elaboratori di testi o app per usi specifici come la gestione di biblioteche). Le specifiche sono controllate da chi sviluppa il software.
- **Prodotti personalizzati:** Sistemi creati su commissione per un cliente specifico (es. sistemi di controllo del traffico aereo). In questo caso, le specifiche sono decise e controllate dal cliente.

Un software ben progettato non deve solo funzionare, ma deve possedere quattro caratteristiche essenziali:

1. **Manutenibilità:** Deve poter evolvere per soddisfare le esigenze dei clienti che cambiano nel tempo.
2. **Affidabilità e sicurezza:** Non deve causare danni economici o fisici in caso di guasto e deve essere protetto da utenti malintenzionati.

3. **Efficienza:** Non deve sprecare risorse del sistema (come memoria o potenza del processore).
4. **Accettabilità:** Deve essere facile da usare, comprensibile e compatibile con i sistemi degli utenti a cui è destinato.

1. Definizione di Ingegneria del Software

L'ingegneria del software è definita come una **disciplina ingegneristica che si occupa di tutti gli aspetti della produzione di software**. Essere una disciplina ingegneristica significa applicare teorie, metodi e strumenti in modo mirato, risolvendo problemi pratici tenendo conto di vincoli finanziari e organizzativi.

È una materia fondamentale perché l'economia e la società dipendono ormai da sistemi software avanzati. Inoltre, realizzare software applicando questi metodi è molto più economico a lungo termine, specialmente perché i costi di manutenzione ed evoluzione del software superano di gran lunga i costi di sviluppo iniziale.

Tutti i processi software prevedono quattro attività fondamentali:

- **Specificazione:** Definire cosa deve fare il software e i suoi limiti.
- **Sviluppo:** Progettare e programmare il sistema.
- **Validazione:** Verificare che il software corrisponda a ciò che il cliente ha richiesto.
- **Evoluzione:** Modificare il software per rispondere ai cambiamenti del mercato e dei clienti.

Il capitolo individua anche le sfide generali che questa disciplina deve affrontare: l'eterogeneità dei sistemi distribuiti su reti diverse, i rapidi cambiamenti aziendali e sociali, la garanzia di sicurezza e fiducia, e la necessità di gestire scale di grandezza diverse (dai micro-dispositivi ai sistemi globali basati su cloud).

2. La Diversità dei Sistemi e l'Impatto del Web

Non esiste un metodo di ingegneria del software universale, ma le tecniche si adattano al tipo di applicazione. Esistono applicazioni stand-alone, sistemi interattivi basati su transazioni, sistemi di controllo integrati (*embedded*), sistemi batch, sistemi di intrattenimento o sistemi di simulazione. Nonostante la diversità, principi come la gestione del processo di sviluppo, l'affidabilità e il riuso del codice valgono per tutti i sistemi.

L'evoluzione del Web ha cambiato profondamente la disciplina. Oggi il software non si installa quasi più localmente, ma si esegue su server web o tramite **cloud computing**. Questo ha spostato l'attenzione sulla costruzione di sistemi tramite l'assemblaggio di componenti preesistenti e sull'ingegneria orientata ai servizi (servizi Web autonomi).

3. L'Etica nell'Ingegneria del Software

Gli ingegneri del software hanno grandi responsabilità e devono mantenere un comportamento onesto ed eticamente responsabile, che va oltre il semplice rispetto delle leggi. Le aree critiche includono:

- **Riservatezza:** Rispettare i segreti professionali di clienti e datori di lavoro.
- **Competenza:** Non accettare compiti al di fuori delle proprie capacità e non falsificare il proprio livello di abilità.
- **Proprietà intellettuale:** Proteggere copyright, brevetti e il lavoro altrui.
- **Uso improprio:** Non usare le proprie capacità per diffondere virus o accedere abusivamente ai sistemi altrui.

2. Processi software

Un processo software è un insieme di attività correlate che portano alla produzione del software.

Tutti i processi software, indipendentemente dal metodo scelto, devono includere 4 attività fondamentali:

1. **Specificazione:** definire le funzionalità e i limiti operativi.
2. **Sviluppo:** creare materialmente il software.
3. **Validazione:** assicurarsi che il prodotto soddisfi le esigenze del cliente.
4. **Evoluzione:** modificare il software per adattarlo ai cambiamenti nel tempo.

A un livello generale, i processi si dividono in **plan-driven** (dove tutte le attività sono pianificate in anticipo) e **agili** (dove la pianificazione è continua e incrementale).

1. I Modelli di Processo Software

Un modello è una rappresentazione semplificata di un processo:

- **Modello a cascata (Waterfall):** È un approccio *plan-driven* che divide il processo in fasi rigorose e sequenziali (analisi dei requisiti, progettazione, implementazione, test di integrazione, e manutenzione). Una fase non inizia finché la precedente non produce documenti approvati. È un modello rigido, indicato solo per requisiti stabili o per enormi sistemi sviluppati in sedi diverse, dove serve un coordinamento ferreo.
- **Sviluppo incrementale:** Le attività di specificazione, sviluppo e validazione si intrecciano. Il software viene rilasciato tramite versioni successive (incrementi), aggiungendo via via funzionalità. È vantaggioso perché riduce i costi dovuti ai cambiamenti dei requisiti, permette di ottenere subito feedback dal cliente e consegna rapidamente software utile. I difetti? È difficile per i manager misurare i progressi (mancano documenti per ogni

piccola fase) e il codice tende a diventare disordinato col tempo se non curato.

- **Integrazione e configurazione (Orientato al riuso):** Costruisce il sistema assemblando componenti preesistenti, come pacchetti software, librerie o servizi web,. Questo riduce i tempi, i costi e i rischi. Tuttavia, costringe a fare compromessi sui requisiti (per adattarsi a ciò che c'è sul mercato) e toglie il pieno controllo sull'evoluzione futura del sistema.

2. Le 4 Attività di Processo

Le quattro attività fondamentali vengono esplorate più a fondo:

- **Specificazione (Ingegneria dei requisiti):** Serve a produrre un documento concordato. Si divide in: elicitazione e analisi (scoprire cosa serve parlando con gli utenti), specificazione (tradurre le info in un documento) e validazione (verificare che i requisiti siano realistici e coerenti),.
- **Progettazione e implementazione:** La progettazione definisce la struttura. Include il design architetturale (struttura generale), del database, delle interfacce e dei componenti,,. Segue poi la programmazione e la rimozione degli errori (debugging).
- **Validazione (Verifica e Validazione - V&V):** Dimostra che il sistema fa ciò che il cliente vuole. Solitamente segue tre step: *testing dei componenti* (moduli isolati), *testing del sistema* (interazioni e interfacce) e *testing del cliente/utente* (valutazione finale con dati reali),. Nei processi *plan-driven* è spesso usato il "modello a V", dove i test vengono pianificati già durante le fasi di progettazione.
- **Evoluzione:** Pochissimi software sono creati da zero; l'ingegneria del software è un continuum dove sviluppo e manutenzione si fondono per rispondere in modo flessibile alle esigenze,.

3. Affrontare il Cambiamento

Il cambiamento è inevitabile e comporta costi di "rielaborazione",. I processi usano due approcci: l'anticipazione o la tolleranza al cambiamento,. Per gestirlo praticamente, si usano due tecniche:

- **Prototipazione:** Si sviluppa rapidamente una versione preliminare e incompleta per validare i requisiti o testare la fattibilità di un design (anticipazione),. Il prototipo va poi scartato, in quanto privo degli standard di qualità e sicurezza di un vero sistema.
- **Consegna incrementale:** Le funzioni del sistema vengono ordinate per priorità. Gli incrementi più importanti vengono sviluppati e consegnati per primi (tolleranza),. In questo modo, le parti critiche ricevono più test, il cliente ottiene valore immediato e i cambiamenti futuri sono più facili,. Il problema è che spesso è difficile identificare le funzioni di base fin dall'inizio e

questo approccio si scontra coi classici contratti che chiedono requisiti completi da subito,.

4. Miglioramento dei Processi

Le aziende software cercano costantemente di migliorare i loro processi per aumentare la qualità o ridurre tempi e costi. Questo si fa in tre fasi cicliche: *misurazione* (raccogliere dati), *analisi* (scoprire i punti deboli) e *cambiamento* (introdurre soluzioni),.

Un approccio storico è quello della "Maturità del processo", in particolare il **modello CMM** (Capability Maturity Model) del SEI, che valuta la maturità di un'azienda su 5 livelli:

1. **Iniziale:** Gli obiettivi base sono raggiunti, ma il lavoro è ancora informale.
2. **Gestito:** Ci sono politiche aziendali standard su quando e come usare i processi.
3. **Definito:** C'è una standardizzazione organizzativa.
4. **Gestito quantitativamente:** Vengono usati metodi statistici e quantitativi per controllare il lavoro.
5. **Ottimizzante:** Livello massimo, dove le misurazioni guidano un miglioramento continuo automatico

3. I Metodi Agili

L'obiettivo dei metodi agili è ridurre al minimo il peso della documentazione inutile e concentrarsi sulla produzione rapida di software funzionante. La filosofia di base è racchiusa nel **Manifesto Agile**, che stabilisce di dare più valore a:

- Individui e interazioni *più che a* processi e strumenti.
- Software funzionante *più che a* documentazione esaustiva.
- Collaborazione col cliente *più che alla* negoziazione dei contratti.
- Rispondere ai cambiamenti *più che a* seguire un piano.

Tutti i metodi agili condividono alcuni **principi fondamentali**: il coinvolgimento continuo del cliente (che stabilisce le priorità), la consegna del software per incrementi, il dare libertà e fiducia alle persone del team, la propensione ad accogliere il cambiamento (invece di opporvisi) e il mantenimento della massima semplicità nel codice.

2. Tecniche di Sviluppo (L'Extreme Programming - XP)

L'approccio che ha maggiormente cambiato la cultura dello sviluppo è stato l'**Extreme Programming (XP)**, ideato da Kent Beck. Si chiama "estremo" perché porta all'estremo le buone pratiche dello sviluppo iterativo (ad esempio, integrare e

testare nuove versioni del software più volte in un solo giorno). Le tecniche principali dell'XP sono:

- **Storie degli utenti (User Stories):** I requisiti non sono scritti in lunghi documenti, ma descritti come "storie" o scenari d'uso scritti su carte. Il team stima il tempo per realizzarle e il cliente decide quali storie implementare per prime in base al valore aziendale.
- **Refactoring continuo:** A differenza dei metodi tradizionali (dove si progetta in anticipo in vista di futuri cambiamenti), in XP si migliora continuamente e immediatamente il codice non appena se ne nota l'occasione. Questo evita che la struttura del software degradi col tempo.
- **Sviluppo Test-First (TDD):** I test vengono scritti *prima* del codice stesso. Si usa un framework di testing automatico (es. JUnit): il test inizialmente fallisce, dopodiché si scrive il codice strettamente necessario per farlo passare. Questo assicura chiarezza sui requisiti e crea una suite di test che previene l'introduzione di nuovi bug in futuro.
- **Pair Programming (Programmazione in coppia):** I programmatori lavorano a due a due su uno stesso computer, scambiandosi spesso di partner. Questo favorisce la proprietà collettiva del codice (tutti conoscono tutto), funge da revisione continua e riduce i rischi se un membro lascia il team.

3. Gestione Agile dei Progetti (Scrum)

Mentre l'XP si concentra sulle tecniche di programmazione, lo **Scrum** è un framework incentrato sulla *gestione* del progetto iterativo. Si divide in tre fasi: pianificazione iniziale, cicli di sviluppo (chiamati *sprint*), e chiusura.

I concetti chiave e la terminologia di Scrum includono:

- **Sprint:** Un ciclo di lavoro a tempo fisso (di solito 2-4 settimane) in cui si sviluppa un incremento del software.
- **Product Backlog:** L'elenco di tutte le cose da fare nel progetto (funzionalità, requisiti, bug da risolvere).
- **Product Owner:** La persona (solitamente in rappresentanza del cliente) che decide quali funzionalità sono prioritarie nel backlog.
- **ScrumMaster:** Un facilitatore che assicura che il team segua le regole di Scrum e lo protegge dalle distrazioni e interferenze esterne.
- **Riunione Scrum (Daily Stand-up):** Un breve incontro quotidiano in cui tutto il team condivide i progressi, il lavoro del giorno e gli eventuali ostacoli.

I vantaggi di Scrum sono enormi: requisiti instabili non bloccano il lavoro, c'è un'alta visibilità su tutto ciò che succede, il cliente vede consegne puntuali di software utile e si crea un forte clima di fiducia.

4. Scalabilità dei Metodi Agili

Se i metodi agili sono perfetti per piccoli team (fino a 7 persone) co-localizzati nello stesso ufficio, applicarli a grandi aziende o a sistemi software enormi è molto difficile. Esistono due sfide principali: lo *"scaling up"* (usare i metodi agili per costruire sistemi giganteschi) e lo *"scaling out"* (introdurre questi metodi in grandi aziende con una mentalità burocratica).

Perché è difficile scalarli?

1. **Contratti:** Le grandi aziende usano contratti legali molto rigidi basati su requisiti specifici predefiniti. L'agile, non avendo specifiche fisse dall'inizio, è incompatibile con questo modello legale (il cliente deve accettare di pagare per il "tempo" impiegato, non per funzioni precise garantite).
2. **Sistemi di Sistemi:** I grandi progetti spesso consistono in più sistemi collegati (es. hardware e software) e vincolati da norme esterne (es. sistemi critici per la sicurezza) che richiedono una rigorosa documentazione preventiva che l'agile di solito evita.
3. **Team Distribuiti:** L'agile prospera sulla comunicazione faccia a faccia, ma le grandi aziende spesso hanno team distribuiti in nazioni e fusi orari diversi.

Come bilanciare approcci agili e plan-driven: Spesso bisogna trovare un compromesso. Si deve usare un **metodo plan-driven** (tradizionale) se il sistema è grande, se ha una lunga vita attesa (richiede molta documentazione per i futuri manutentori), se è soggetto a certificazioni legali/regolamentari o se il team non ha grandi competenze. Se si cerca di usare l'agile su larga scala (come nel modello elaborato da IBM o con lo "Scrum of Scrums"), è necessario introdurre dei ruoli come gli *Architetti del prodotto* per coordinare il design globale, e accettare l'idea che non si può fare a meno di una certa pianificazione e documentazione iniziale. Infine, introdurre l'agile nelle grandi aziende spesso incontra la resistenza culturale dei manager abituati a procedure rigorose e standard di qualità burocratici.

4. Requisiti

1. Cosa sono i requisiti e chi sono gli Stakeholder

Un requisito è la descrizione di un servizio che il sistema deve fornire o di un vincolo legato al suo funzionamento. I requisiti non sono scritti tutti allo stesso modo, ma variano in base a chi li deve leggere:

- **Requisiti utente:** Sono dichiarazioni astratte e ad alto livello (scritte in linguaggio naturale e con diagrammi) che descrivono i servizi offerti agli utenti e i vincoli operativi. Sono pensati per essere letti da clienti, manager e utenti finali, che non hanno necessariamente competenze tecniche.
- **Requisiti di sistema:** Sono descrizioni molto più dettagliate e tecniche delle funzioni del software. Spesso fanno parte del contratto tra cliente e

sviluppatore e servono agli ingegneri e ai programmatori per capire esattamente cosa implementare.

Le persone coinvolte o influenzate dal sistema sono chiamate **stakeholder**. Questi includono utenti finali, manager, ingegneri, ma anche figure esterne come i legislatori o gli enti regolatori.

2. Requisiti Funzionali e Non Funzionali

Questa è una distinzione fondamentale nell'ingegneria del software:

- **Requisiti Funzionali:** Descrivono *cosa* il sistema deve fare. Indicano i servizi offerti, come il sistema deve reagire a specifici input o come deve comportarsi in determinate situazioni (es. "L'utente deve poter cercare la lista degli appuntamenti"). Idealmente, dovrebbero essere completi (definire tutti i servizi) e coerenti (senza contraddizioni interne), anche se nella pratica è molto difficile.
- **Requisiti Non Funzionali:** Non riguardano le singole funzioni, ma sono *vincoli* sull'intero sistema (es. tempi di risposta, affidabilità, uso della memoria, standard da rispettare). Sono spesso più critici di quelli funzionali: se una funzione manca, l'utente può trovare un'alternativa, ma se il sistema è troppo lento o insicuro, diventa completamente inutilizzabile. Si dividono in tre categorie:
 1. *Requisiti di prodotto:* Comportamento del software (es. velocità, spazio).
 2. *Requisiti organizzativi:* Procedure aziendali o standard di sviluppo.
 3. *Requisiti esterni:* Leggi, normative sulla privacy o etica.
- **Regola d'oro:** I requisiti non funzionali **non dovrebbero esprimere vaghi "obiettivi"** (es. "Il sistema deve essere facile da usare"), ma devono essere quantificabili e testabili oggettivamente (es. "Il numero medio di errori di un utente esperto non deve superare i due all'ora").

3. Il Processo di Ingegneria dei Requisiti

Il processo non è lineare, ma iterativo (spesso rappresentato come una spirale) e si compone di tre fasi principali che si intrecciano: elicitazione, specificazione e validazione.

A. Elicitazione (Scoperta e Analisi)

È la fase in cui gli ingegneri collaborano con gli stakeholder per scoprire cosa serve. È molto difficile perché gli stakeholder spesso non sanno cosa vogliono, usano un gergo incomprensibile agli informatici, hanno esigenze in conflitto tra loro o cambiano idea per via del mercato dinamico. Le tecniche usate per scoprire i requisiti sono:

- **Interviste:** Possono essere chiuse (domande predefinite) o aperte (discussioni libere). Ottime per il quadro generale, ma meno efficaci per i dettagli tecnici.

- **Etnografia:** Consiste nell'osservare le persone mentre lavorano. È utilissima per scoprire i requisiti impliciti e capire come le persone lavorano *realmente*, al di là delle procedure aziendali formali.
- **Storie e Scenari (Use Cases):** Le persone comprendono meglio gli esempi pratici. Uno scenario descrive una situazione reale di utilizzo, specificando la situazione di partenza, il flusso normale degli eventi, cosa può andare storto e lo stato finale del sistema.

B. Specificazione

È la traduzione delle informazioni raccolte in un documento formale e strutturato. I metodi di scrittura includono:

- **Linguaggio naturale:** Frasi normali. È intuitivo e universale, ma rischia di essere ambiguo. Per migliorarlo, si usano formati standard, formattazione (grassetto/corsivo) ed è fondamentale evitare il gergo tecnico e spiegare sempre il *perché* di un requisito.
- **Specifiche strutturate:** Si usano modelli predefiniti (moduli) in cui si indicano esplicitamente input, output, azioni, pre-condizioni e post-condizioni. Spesso si integrano con tabelle logiche per spiegare calcoli complessi (es. il calcolo della dose in una pompa di insulina).
- **Casi d'uso (UML):** Modelli grafici che mostrano le interazioni tra gli "attori" (utenti o altri sistemi) e il sistema. Tutto questo confluisce nel **Documento dei Requisiti Software (SRS)**, che è la dichiarazione ufficiale di ciò che andrà implementato.

C. Validazione

Questa fase serve a verificare che i requisiti definiti siano *esattamente* ciò che il cliente vuole. È una fase cruciale: correggere un errore nei requisiti a sviluppo inoltrato costa moltissimo, perché costringe a rifare progettazione, codice e test. Durante la validazione si controlla:

- **Validità:** Soddisfa i bisogni reali?
- **Coerenza e Completezza:** Ci sono contraddizioni? Manca qualcosa?
- **Realismo e Verificabilità:** Si può fare col budget/tecnologia attuale? I requisiti si possono testare? Si usano tecniche come le *revisioni sistematiche* (da parte di un team), la creazione di *prototipi* e la scrittura anticipata di *casi di test*.

4. Gestione dei Requisiti (Cambiamento)

I requisiti di un grande sistema cambieranno sempre, perché cambia il mercato, cambiano le leggi o semplicemente perché gli stakeholder cambiano le loro priorità nel tempo.

La **gestione dei requisiti** è il processo che tiene traccia di queste modifiche. Per farlo bene serve assegnare un ID univoco a ogni requisito (per la

tracciabilità) e usare strumenti software appositi. Quando viene proposta una modifica, si segue un processo in 3 step:

1. **Analisi del problema:** Si valuta se la modifica è valida e necessaria.
2. **Analisi dei costi:** Si valuta l'impatto della modifica sugli altri requisiti e quanto costerà implementarla.
3. **Implementazione:** Se approvata, si aggiornano i documenti, il design e il codice.

(Nota: a differenza dei metodi tradizionali descritti fin qui, i metodi "Agili" considerano la scrittura di un documento dettagliato una perdita di tempo, preferendo gestire i requisiti in modo incrementale scrivendoli sotto forma di "storie degli utenti").

5. Modellazione del sistema

La modellazione si avvale solitamente del linguaggio grafico **UML** (Unified Modeling Language) e serve per discutere i requisiti, documentare i sistemi esistenti o, in alcuni casi, generare direttamente il codice.

5 fondamentali (attività, casi d'uso, sequenza, classi e stato) per esplorare quattro diverse prospettive del sistema:

1. Modelli di Contesto (Prospettiva Esterna)

Nelle fasi iniziali, è fondamentale definire i **confini del sistema**, ovvero decidere cosa farà parte del software e cosa sarà gestito manualmente o da altri sistemi esterni. Questa decisione è spesso influenzata da fattori sociali e organizzativi. I modelli di contesto mostrano il sistema al centro, circondato dagli altri sistemi con cui interagisce (ad esempio, il sistema clinico *Mentcare* che si interfaccia con il sistema delle ammissioni o delle prescrizioni). Tuttavia, poiché questi modelli non spiegano *come* avvengono le interazioni, vengono spesso affiancati dai **diagrammi delle attività UML**, che illustrano i processi aziendali più ampi, il flusso di lavoro e le attività umane e automatizzate.

2. Modelli di Interazione (Prospettiva di Interazione)

Modellare le interazioni (tra utente e sistema, o tra componenti del sistema stesso) è vitale per identificare i requisiti e i colli di bottiglia. Si usano due diagrammi principali:

- **Diagrammi dei casi d'uso:** Forniscono una panoramica semplice. Un "caso d'uso" (un'ellisse) rappresenta un'attività, mentre gli "attori" (figure stilizzate) sono gli utenti o i sistemi esterni coinvolti. Poiché sono molto stilizzati, devono essere accompagnati da descrizioni testuali dettagliate (input, output, stimoli e risposte).

- **Diagrammi di sequenza:** Mostrano l'ordine cronologico esatto delle interazioni per un singolo caso d'uso. Si leggono dall'alto verso il basso lungo delle "linee di vita" (lifeline) tratteggiate che rappresentano gli oggetti o gli attori coinvolti. Le frecce indicano i messaggi o le chiamate ai metodi scambiati tra di loro, e appositi riquadri (chiamati "alt") mostrano percorsi alternativi, come in caso di successo o fallimento di un'autorizzazione.

3. Modelli Strutturali (Prospettiva Strutturale)

Questi modelli descrivono l'organizzazione e l'architettura del software. L'UML usa i **diagrammi delle classi** per definire i tipi di oggetti presenti nel sistema, i loro attributi (caratteristiche), le loro operazioni (funzioni) e le relazioni (associazioni) tra di loro. Per gestire la complessità strutturale, si usano due tecniche fondamentali:

- **Generalizzazione:** Simile al concetto di ereditarietà nella programmazione orientata agli oggetti. Si creano "classi generali" (superclassi) e "classi specifiche" (sottoclassi) che ereditano attributi e metodi da quelle superiori (es. la classe generale "Medico" si divide in "Medico Ospedaliero" e "Medico di Base"). Questo facilita le modifiche future, evitando di dover aggiornare ogni singola classe.
- **Aggregazione:** Mostra una relazione di tipo "tutto-parte", dove un oggetto principale è composto da altri oggetti più piccoli.

4. Modelli Comportamentali (Prospettiva Comportamentale)

Mostrano cosa succede dinamicamente quando il sistema è in esecuzione e risponde a degli stimoli. Gli stimoli si dividono in due categorie, gestite con modelli diversi:

- **Modelli guidati dai dati:** Usati tipicamente nei sistemi aziendali (come l'elaborazione delle fatture), mostrano la sequenza end-to-end con cui un dato in ingresso viene trasformato fino all'output. Si rappresentano tramite i diagrammi di attività.
- **Modelli guidati dagli eventi:** Tipici dei sistemi in tempo reale (come un forno a microonde), dove il sistema deve rispondere a input esterni imprevisti. Si usano i **diagrammi di stato**, che partono dal presupposto che il sistema abbia un numero finito di "stati". Il diagramma mostra come un evento (es. "porta aperta") forzi la transizione da uno stato all'altro (es. da "in operazione" a "disabilitato"). Per evitare che il diagramma diventi illeggibile per via dei troppi stati, si usano i "superstati", che incapsulano al loro interno una serie di sotto-stati.

5. Ingegneria Guidata dai Modelli (MDE)

L'ultima parte del capitolo affronta un approccio moderno in cui i modelli UML non sono solo disegni preparatori, ma diventano il prodotto principale da cui **il codice eseguibile viene generato automaticamente** tramite appositi strumenti.

L'iniziativa più nota in questo ambito è l'**Architettura guidata da modelli (MDA)**, che prevede tre passaggi:

1. **CIM (Computation Independent Model)**: Un modello di dominio ad altissimo livello, indipendente dal calcolo.
2. **PIM (Platform-Independent Model)**: Un modello del funzionamento del sistema che non fa riferimento a nessuna tecnologia specifica.
3. **PSM (Platform-Specific Model)**: Il PIM viene tradotto automaticamente in modelli specifici per le varie piattaforme (es. uno per Java e uno per .NET) e infine in codice eseguibile.

Tuttavia, il testo specifica che questo approccio fatica a imporsi universalmente. I motivi principali sono: i modelli ottimi per discutere non sono sempre adatti per generare codice; il vero scoglio dello sviluppo non è scrivere codice ma capire i requisiti; infine, l'idea di dover fare tutta questa modellazione complessa all'inizio del progetto è in forte conflitto con la filosofia flessibile dei metodi Agili.

6. Architectural design

Anche nei metodi Agili (che tendono a rimandare le decisioni), si accetta che l'architettura generale debba essere definita nelle fasi iniziali. Il motivo è semplice: sebbene sia facile modificare (rifattorizzare) singoli pezzetti di codice, modificare l'intera architettura del sistema a progetto avviato è estremamente costoso, perché costringe a riscrivere quasi tutti i componenti.

1. Livelli di Architettura e i suoi Vantaggi

L'architettura software può essere pensata su due livelli di astrazione:

- **Architettura "in piccolo" (in the small)**: Riguarda i singoli programmi e come sono suddivisi in componenti.
- **Architettura "in grande" (in the large)**: Riguarda sistemi complessi o aziendali che includono altri sistemi, distribuiti su più computer e persino gestiti da aziende diverse.

Avere un'architettura progettata e documentata esplicitamente porta tre grandi vantaggi:

1. **Comunicazione**: Offre una visione ad alto livello che tutti gli stakeholder possono usare per discutere il progetto.
2. **Analisi del sistema**: Permette di valutare fin da subito se il sistema riuscirà a soddisfare requisiti critici (come prestazioni e affidabilità).
3. **Riutilizzo su larga scala**: Sistemi con requisiti simili spesso usano la stessa architettura, facilitando il riuso di intere strutture.

2. Guidare l'Architettura con i Requisiti Non Funzionali

Non esiste una formula magica per creare un'architettura. La scelta dipende strettamente dai **requisiti non funzionali** che il sistema deve garantire:

- **Prestazioni:** Se il sistema deve essere veloce, conviene avere pochi componenti di grandi dimensioni per ridurre le comunicazioni tra di essi.
- **Sicurezza:** Conviene usare un'architettura a livelli (a strati), posizionando le risorse più critiche nei livelli più interni, protetti da rigidi controlli.
- **Sicurezza operativa (Safety):** Le operazioni vitali devono essere concentrate in pochissimi componenti per minimizzare il rischio di guasti.
- **Disponibilità:** Se il sistema non deve mai spegnersi, servono componenti ridondanti (duplicati) da sostituire senza interrompere il servizio.
- **Manutenibilità:** Si devono usare componenti piccoli, indipendenti e facili da modificare.

3. Le Viste Architetture (Il modello 4+1)

Poiché è impossibile mostrare tutte le informazioni di un sistema in un solo disegno, si usano diverse "viste" o prospettive. Il modello più famoso è il **"4+1 view" di Krutchen**:

1. **Vista logica:** Mostra gli oggetti chiave o le classi del sistema.
2. **Vista di processo:** Mostra come i processi interagiscono tra loro a runtime (mentre il programma è in esecuzione).
3. **Vista di sviluppo:** Mostra come il software è diviso in moduli per facilitare il lavoro dei vari team di programmatori.
4. **Vista fisica:** Mostra l'hardware e come il software è distribuito sui processori.
5. **+1 (Casi d'uso):** Scenari che collegano tutte le quattro viste precedenti.

4. I Pattern Architetture

I pattern architetturali sono descrizioni standardizzate di buone pratiche organizzative che hanno già funzionato in passato. Il capitolo ne analizza cinque fondamentali:

- **MVC (Model-View-Controller):** Separa i dati del sistema (Model) dalla loro presentazione all'utente (View) e dalla gestione delle interazioni (Controller). È lo standard per le applicazioni web, perché permette di mostrare gli stessi dati in formati diversi senza cambiare il cuore del programma. Svantaggio: introduce codice inutile se l'interfaccia è molto semplice. In interfacce molto semplici, spesso il Controller finisce per fare solo da "passacarte": riceve i dati dal Model e li passa identici alla View senza fare alcuna elaborazione. In questo caso, il Controller è un pezzo di codice che esiste solo per rispettare la regola dell'MVC, ma non serve a nulla dal punto di vista pratico.

- **Architettura a livelli (Layered):** Il sistema è diviso in strati. Ogni strato offre servizi solo al livello superiore e usa i servizi di quello inferiore. Vantaggio: puoi sostituire un intero livello (es. cambiare il database) senza intaccare il resto, e supporta lo sviluppo incrementale. Svantaggio: **le prestazioni possono calare perché i dati devono attraversare molti strati.**
- **Architettura a Repository:** C'è un database centrale (repository) a cui tutti i componenti accedono. I componenti non comunicano tra loro direttamente. Ottimo quando si generano grandi volumi di dati, come negli ambienti di sviluppo IDE. Svantaggio: se il repository centrale si guasta, crolla l'intero sistema (single point of failure).
- **Architettura Client-Server:** Le funzionalità sono offerte sotto forma di servizi (i server), e gli utenti (i client) vi accedono tramite rete. Eccellente per database condivisi e per bilanciare i carichi di lavoro distribuendo i server. Svantaggio: dipendenza dalla rete e vulnerabilità dei server. Se il server viene attaccato è un gran casino...
- **Architettura Pipe and Filter (Filtri e Condotti):** I dati fluiscono in un tubo (pipe) passando attraverso vari componenti (filtri). Ogni filtro trasforma il dato in un formato utile per il filtro successivo. È il modello classico per l'elaborazione dei dati in batch (es. elaborare un blocco di fatture e poi emettere i pagamenti). Svantaggio: tutti i filtri devono accordarsi su un formato di dati comune, aumentando il sovraccarico del sistema per la conversione continua dei dati.

5. Architetture Applicative Generiche

I sistemi usati dalle aziende (sistemi applicativi) spesso si assomigliano: ad esempio, le banche hanno tutte esigenze simili. Si usano quindi dei "modelli standard" come base di partenza. Il testo ne illustra due tipologie:

- **Sistemi di elaborazione delle transazioni:** Gestiscono richieste interattive e asincrone degli utenti per aggiornare o consultare un database mantenendone l'integrità. Esempi tipici sono bancomat, e-commerce o sistemi di prenotazione. Seguono spesso una struttura del tipo: input (es. inserisco bancomat) -> elaborazione logica -> gestore transazioni (database) -> output (es. prelievo). I sistemi e-commerce moderni usano una versione di questa chiamata "multi-tier" (server web, server applicativo, server database).
- **Sistemi di elaborazione del linguaggio:** Traducono un linguaggio formale in istruzioni macchina (es. i compilatori). Trasformano il codice sorgente passando per vari step: analisi lessicale, sintattica, semantica e generazione di codice. Architetaturalmente, sono spesso un ibrido: usano un *Repository* (come tabella dei simboli condivisa) unito al modello *Pipe and filter* (le varie fasi di analisi che si passano il codice).

7. Test del software

1. Gli Obiettivi del Testing e la V&V (Verifica e Validazione)

Il testing consiste nell'eseguire un programma utilizzando dati artificiali per verificare i risultati e individuare errori o anomalie. Testare il software ha due obiettivi principali:

1. **Testing di validazione:** Dimostrare al cliente e al team che il software soddisfa i requisiti richiesti (ci deve essere almeno un test per ogni requisito).
2. **Testing per la rilevazione dei difetti:** Trovare input che generano comportamenti scorretti, crash o corruzione dei dati per esporre e correggere i bug.

Queste attività rientrano nel più ampio processo di **V&V (Verifica e Validazione)**, due concetti che spesso vengono confusi ma che sono distinti:

- **Validazione:** "Stiamo costruendo il prodotto giusto?". Assicura che il software risponda alle reali aspettative del cliente.
- **Verifica:** "Stiamo costruendo il prodotto nel modo giusto?". Controlla che il software sia conforme alle specifiche tecniche definite.

L'obiettivo della V&V è stabilire che il sistema sia "adatto allo scopo", il cui livello dipende dalla criticità del software, dalle aspettative degli utenti e dall'ambiente di mercato.

Oltre al testing dinamico (eseguire il codice), la V&V include tecniche "statiche" come le **ispezioni**. Le ispezioni analizzano il codice, i modelli o i requisiti senza eseguire il software. I vantaggi delle ispezioni sono che un errore non nasconde altri errori, possono essere applicate a sistemi incompleti e valutano anche la qualità del codice (es. manutenibilità). Tuttavia, non sostituiscono il testing, poiché non possono rilevare problemi legati alle interazioni impreviste o alle prestazioni in tempo reale.

Il testing di un software commerciale si divide in tre fasi principali: sviluppo, rilascio e utente.

2. Test di Sviluppo (Development Testing)

Questi test sono eseguiti dal team di sviluppo per scoprire bug e difetti durante la creazione del software. Si dividono in tre sotto-fasi:

A. Unit Testing (Testing dell'unità)

Consiste nel testare singoli componenti, come metodi, funzioni o classi di oggetti. Quando si testa un oggetto, bisogna testare tutte le sue operazioni, controllare i suoi attributi e metterlo in tutti gli stati possibili. Idealmente, l'unit testing dovrebbe essere **automatizzato** usando framework come JUnit. Un test automatizzato si divide in tre parti: *configurazione* (inizializzazione degli input), *chiamata* (esecuzione dell'oggetto) e *asserzione* (confronto tra il risultato ottenuto e quello atteso).

Poiché testare tutte le combinazioni è impossibile, si usano due strategie per **scegliere i casi di test**:

- **Testing per partizione (Partizioni di equivalenza)**: Gli input e gli output vengono raggruppati in "famiglie" che il programma dovrebbe elaborare allo stesso modo. Si scelgono casi di test ai confini di queste partizioni e alcuni nel punto medio.
- **Testing basato su linee guida**: Si usa l'esperienza dei programmatori sugli errori più comuni. Esempi di linee guida includono: testare array con un solo valore, forzare l'overflow dei buffer, inserire dati che generino messaggi di errore o forzare risultati matematici troppo grandi/piccoli.

B. Component Testing (Testing dei componenti)

In questa fase si integrano più unità per formare componenti più grandi. L'obiettivo qui non è ritestare le singole funzioni (già testate nell'unit test), ma **testare le interfacce** attraverso cui i componenti comunicano. Esistono vari tipi di interfacce (a parametri, a memoria condivisa, procedurali, a passaggio di messaggi) e i problemi più comuni sono l'uso improprio dell'interfaccia, incomprensioni sul suo funzionamento o errori di sincronizzazione nei tempi di esecuzione. Per testarle, si consiglia di usare valori estremi per i parametri, testare i puntatori nulli e usare lo *stress testing* per sovraccaricare il sistema.

C. Testing del Sistema

Tutti i componenti (inclusi quelli preesistenti o commerciali) vengono integrati e il sistema viene testato nella sua interezza. A differenza dei test precedenti, questo è un processo collettivo che verifica la compatibilità globale. Un approccio molto efficace in questa fase è il **testing basato sui casi d'uso**, che forza le interazioni tra i vari oggetti del sistema. Poiché il testing esaustivo è impossibile, ci si concentra su sottoinsiemi critici, testando ad esempio tutte le funzioni dei menu con input corretti e scorretti.

3. Test-Driven Development (TDD)

Lo **Sviluppo Guidato dai Test** è un approccio in cui la scrittura dei test avviene *prima* di scrivere il codice. Il ciclo funziona così: si identifica una piccola funzionalità da aggiungere, si scrive il test automatizzato, lo si esegue (ovviamente fallirà perché il codice non esiste ancora), si implementa il codice strettamente necessario e si riesegue il test finché non viene superato. I vantaggi del TDD includono: una copertura del codice totale, un debugging molto semplificato e un'eccellente documentazione. Il vantaggio più grande, tuttavia, è la riduzione drastica dei costi del **testing di regressione** (ovvero il test che verifica se le nuove modifiche hanno rotto il codice preesistente), poiché tutti i test vengono rieseguiti in automatico a ogni modifica.

4. Testing di Rilascio (Release Testing)

Questa fase testa la versione completa prima che venga consegnata ai clienti. A differenza dei test di sviluppo (che cercano bug), il testing di rilascio è un processo di verifica per dimostrare che il sistema è "sufficientemente buono" e soddisfa i requisiti, le prestazioni e l'affidabilità. Deve essere eseguito da un team di test separato dagli sviluppatori e si basa su un approccio *black-box* (il sistema è visto come una scatola nera di cui si guardano solo input e output). Si divide in:

- **Testing basato sui requisiti:** Si crea almeno un test per ogni requisito specificato nel documento ufficiale (es. verificare che scatti un allarme se si prescrive un farmaco a cui il paziente è allergico).
- **Testing basato su scenari:** Si inventa una "storia" realistica di utilizzo quotidiano (es. la giornata tipo di un infermiere che scarica i dati, visita i pazienti, segna gli effetti collaterali e ricarica i dati) per derivare i casi di test.
- **Testing delle prestazioni e Stress Testing:** Si aumenta progressivamente il carico di lavoro sul sistema fino a superare il limite, per verificare che gestisca la mole di dati prevista e per osservarne il comportamento in caso di guasto sotto sforzo.

5. Testing dell'Utente (User Testing)

Anche dopo test rigorosi, il testing dell'utente è vitale perché gli sviluppatori non possono mai replicare esattamente il vero ambiente di lavoro (es. le interruzioni o le emergenze in un ospedale). Esistono tre tipi di test dell'utente:

1. **Test Alpha:** Un gruppo selezionato di utenti lavora a stretto contatto con gli sviluppatori per testare le prime versioni.
2. **Test Beta:** La versione viene distribuita a un pubblico più ampio che sperimenta il software e segnala i problemi.
3. **Test di Accettazione:** I clienti testano il sistema finale per decidere se accettarlo (e pagarlo) o rifiutarlo.

Il processo di test di accettazione prevede 6 fasi: definire i criteri di accettazione (idealmente prima di firmare il contratto), pianificare le risorse, progettare i test, eseguire i test nell'ambiente reale, negoziare i risultati (se ci sono problemi) e infine decidere per il rifiuto o l'accettazione. Nei metodi agili, questo processo è integrato: l'utente fa parte del team e i test basati sulle sue *user stories* fungono direttamente da test di accettazione continui. L'unica criticità è che a volte è difficile trovare utenti che siano veramente "tipici".

8. Pianificazione del progetto

Il processo di pianificazione attraversa tre fasi principali:

1. **Fase di proposta:** Si elabora un piano speculativo per decidere se si hanno le risorse per il lavoro e per calcolare il prezzo da proporre al cliente nella gara d'appalto.
2. **Fase di avvio del progetto:** Ottenuto il contratto, si pianifica in dettaglio chi lavorerà, come suddividere gli incrementi e come allocare le risorse.
3. **Aggiornamenti periodici:** Il piano viene continuamente aggiornato durante il progetto per riflettere nuove informazioni su software, tempi, costi e rischi.

Di seguito, l'analisi dettagliata di tutti i concetti del capitolo.

1. Prezzi del Software

In teoria, il prezzo di un software per un cliente è il costo di sviluppo più il profitto. In pratica, entrano in gioco fattori commerciali, organizzativi ed economici che possono alzare o abbassare il prezzo. I fattori chiave includono:

- **Termini contrattuali:** Se lo sviluppatore mantiene la proprietà del codice sorgente per riutilizzarlo, il prezzo può essere inferiore.
- **Incertezza sui costi:** Se i costi sono incerti o si usa un contratto a prezzo fisso, l'azienda aggiunge una percentuale di contingenza, alzando il prezzo.
- **Salute finanziaria e opportunità di mercato:** Un'azienda in difficoltà può abbassare il prezzo pur di mantenere il flusso di cassa. Allo stesso modo, può farlo per entrare in un nuovo segmento di mercato.
- **Volatilità dei requisiti:** Se i requisiti cambieranno spesso, si può offrire un prezzo base basso per vincere il contratto, applicando poi prezzi alti per ogni successiva modifica. A volte si utilizza la strategia del "*pricing to win*", in cui l'azienda propone il prezzo che pensa il cliente sia disposto a pagare, negoziando poi le specifiche del progetto in base a quel budget.

2. Sviluppo Plan-Driven (Guidato dal Piano)

Nello sviluppo tradizionale "plan-driven", tutto il processo viene pianificato in dettaglio fin dall'inizio. Il **piano di progetto** documenta risorse, suddivisione del lavoro e un programma (schedule). Le sezioni tipiche di un piano includono: introduzione, organizzazione del progetto, analisi dei rischi, requisiti hardware/software, suddivisione del lavoro, pianificazione delle attività e meccanismi di monitoraggio. A questo si affiancano piani supplementari come quelli per la qualità, la manutenzione, la validazione e la gestione delle configurazioni.

Il **processo di pianificazione** è iterativo. Si inizia valutando i vincoli (budget, personale, date) e definendo i traguardi. Il piano va poi costantemente rivisto. Se emergono problemi gravi che causano ritardi, si attuano "azioni di mitigazione del rischio" e si procede alla ripianificazione, che può comportare anche la rinegoziazione dei vincoli con il cliente. È vitale fare ipotesi realistiche (e non ottimistiche) e includere sempre un margine di "contingenza".

3. Pianificazione e Programmazione delle Attività

Pianificare significa suddividere il lavoro in compiti separati (tasks). Una regola d'oro è che **un compito dovrebbe durare da 1 a 8 settimane** (massimo 2 mesi). Compiti più brevi richiedono troppa micro-gestione, compiti più lunghi vanno ulteriormente suddivisi. Bisogna coordinare i compiti paralleli per ottimizzare la forza lavoro ed evitare dipendenze che blocchino l'intero progetto. Aggiungere personale a un progetto già in ritardo non aiuta, anzi, lo rallenta ulteriormente a causa dei costi di comunicazione tra i membri.

Per presentare la pianificazione si usano due strumenti visivi:

1. **Reti di attività:** Mostrano le dipendenze logiche tra i vari compiti.
2. **Grafici a barre (Diagrammi di Gantt):** Mostrano le date di inizio/fine delle attività, i responsabili e l'allocazione del personale nel tempo (con linee diagonali per indicare il lavoro part-time). La pianificazione identifica anche le **tappe (milestones)**, ovvero la fine logica di una fase per la revisione, e i **deliverable**, che sono i risultati concreti consegnati al cliente.

4. Pianificazione Agile

A differenza del plan-driven, i metodi agili non pianificano tutta la funzionalità in anticipo, ma decidono cosa includere iterazione dopo iterazione, adattandosi ai cambiamenti del cliente. La pianificazione agile si divide in due fasi:

- **Pianificazione del rilascio:** Guarda avanti di diversi mesi per decidere le funzionalità di una release.
- **Pianificazione dell'iterazione:** Guarda a 2-4 settimane di lavoro (lo *sprint*).

In Extreme Programming si usa il "**planning game**" basato sulle "storie degli utenti". Team e cliente discutono le storie e le classificano per sforzo richiesto (usando un punteggio relativo, es. 2 punti o 8 punti). Si calcola poi la **velocità (velocity)**, ovvero i punti implementati dal team in un giorno. Con la velocità si stima lo sforzo totale. All'inizio di un'iterazione, le storie vengono divise in task (da 4 a 16 ore). Un grande vantaggio è che **sono gli sviluppatori stessi a scegliere i compiti**, aumentando motivazione e senso di responsabilità. Se il tempo finisce, il lavoro non completato viene rimandato, ma la data di consegna dell'incremento non slitta mai. Questo approccio funziona bene con team piccoli e stabili, ma fatica a scalare su team grandi o distribuiti.

5. Tecniche di Stima dei Costi

Nelle fasi iniziali, la stima è difficilissima: il margine d'errore può variare da 0,25x a 4x rispetto al costo reale. L'accuratezza migliora solo a progetto avanzato. Le organizzazioni usano due tecniche:

1. **Tecniche basate sull'esperienza:** Il manager stima lo sforzo basandosi su progetti passati simili, magari usando fogli di calcolo e discutendo in gruppo. Il difetto è che se si usano tecnologie nuove (es. nuovi linguaggi o Web Services), l'esperienza passata diventa inutile.
2. **Modelli algoritmici di costo:** Usano formule matematiche basate su metriche del prodotto. La formula base è: $\text{Effort} = A \times \text{Size}^B \times M$. (Dove A è una costante, Size è la dimensione del codice, B è la complessità del progetto e M è un moltiplicatore che include attributi di processo e abilità del team). A causa della loro complessità, sono usati per lo più da grandi industrie (es. difesa/aerospazio).

6. Modellazione dei Costi: COCOMO II

Il modello empirico COCOMO II è il modello algoritmico più famoso. È supportato da strumenti open-source e si adatta a tecniche moderne come il riuso del codice e i linguaggi dinamici. È composto da quattro sottomodelli, usati in fasi via via più avanzate del progetto:

1. **Modello di composizione dell'applicazione:** Usato per i prototipi o per software creato unendo componenti esistenti. Calcola lo sforzo basandosi sugli "application points" divisi per la produttività stimata del team.
2. **Modello di progettazione preliminare:** Usato nelle fasi iniziali per esplorare opzioni. Applica la formula base vista prima ($A \times \text{Size}^B \times M$) utilizzando 7 attributi (es. capacità del personale, difficoltà della piattaforma, programmazione temporale) per calcolare il moltiplicatore M. L'esponente B varia da 1,1 a 1,24 in base alla flessibilità e novità del progetto.
3. **Modello di riuso:** Stima l'impegno per integrare codice preesistente. Distingue tra *codice black-box* (integrato così com'è, sforzo zero) e *codice white-box* (deve essere compreso e modificato, richiede sforzo calcolato tramite una formula specifica).
4. **Livello post-architetturale:** È il modello più dettagliato, usato dopo aver progettato l'architettura. Calcola la dimensione totale sommando nuove righe di codice, codice riutilizzato e codice modificato per cambi di requisiti. L'esponente B viene calcolato usando 5 fattori di scala (Precedenza/novità, Flessibilità, Risoluzione rischi, Coesione team, Maturità processo) da cui si deriva un punteggio. Il moltiplicatore finale usa ben 17 attributi anziché 7 per affinare la stima.

Durata del progetto e gestione del personale: COCOMO fornisce anche una formula per calcolare i mesi di calendario necessari per finire il progetto ($\text{TDEV} = 3 \times (\text{PM}) (0.33 + 0.2 \times (B - 1.01))$). Il modello dimostra matematicamente che la relazione tra personale e tempo non è lineare. Aggiungere personale fa calare la produttività generale (per via del tempo speso in comunicazioni e interfacciamenti), quindi raddoppiare le persone non dimezza mai i tempi.

9. Altri argomenti

Discrete Time Markov Chains (DTMC): Guida Completa

1. Definizione Formale

Una **Catena di Markov a Tempo Discreto (DTMC)** è un modello matematico utilizzato per descrivere sistemi che evolvono attraverso una sequenza di stati in istanti di tempo discreti. La caratteristica fondamentale è la **Proprietà di Markov**: la probabilità di transizione allo stato successivo dipende *esclusivamente* dallo stato attuale e non dalla storia passata del sistema.

Una DTMC è definita dalla quintupla $M=(U,X,Y,p,g)$:

- **U (Insieme di Input)**: Gli stimoli esterni che influenzano le transizioni. Può essere un insieme vuoto (sistema autonomo), finito o infinito.
- **X (Insieme di Stati)**: Lo spazio degli stati (non vuoto). Rappresenta tutte le possibili condizioni in cui può trovarsi il sistema.
- **Y (Insieme di Output)**: I valori osservabili generati dal sistema.
- **p (Funzione di Probabilità di Transizione)**: Definisce la dinamica del sistema.

$$p: X \times X \times U \rightarrow [0,1]$$

Rappresenta la probabilità $p(x'|x,u)$ di passare allo stato x' partendo da x con input u . Deve soddisfare la condizione di normalizzazione:

$$\sum_{x'} p(x'|x,u) = 1 \quad (\text{o } \int X p(x'|x,u) dx' = 1 \text{ per spazi contini})$$

- **g (Funzione di Output)**: Associa a ogni stato un'osservazione misurabile.

$$g: X \rightarrow Y$$

2. Analisi di Processo: Esempi Pratici

Immaginiamo un processo di sviluppo software suddiviso in tre fasi: **Fase 1 (Requisiti)**, **Fase 2 (Sviluppo)**, **Fase 3 (Test)**.

A. Calcolo del Tempo di Completamento

In questo scenario, ogni fase ha una durata specifica. Il tempo totale è la somma delle durate delle fasi attraversate.

Fase	Durata	Transizioni Possibili
F1: Requisiti	2 mesi	Avanti (F2): 0.7
F2: Sviluppo	4 mesi	Avanti (F3): 0.6
F3: Test	2 mesi	Fine: 0.6

Esporta in Fogli

Esempi di calcolo delle probabilità:

1. **Esattamente 8 mesi (Percorso lineare F1→F2→F3→Out):**

$$P(T=8)=1.0 \times 0.7 \times 0.6 \times 0.6 = 0.252$$

2. **Esattamente 10 mesi (Un ritardo di 2 mesi):**

Si verifica se il sistema "stalla" per un ciclo in F1 o in F3.

- Percorso F1(Resta) → F1 → F2 → F3: $0.3 \times 0.7 \times 0.6 \times 0.6 = 0.0756$
- Percorso F1 → F2 → F3(Resta) → F3: $0.7 \times 0.6 \times 0.1 \times 0.6 = 0.0252$
- **Totale:** $0.0756 + 0.0252 = 0.1008$

B. Calcolo del Costo e Parametrizzazione

Sostituendo il tempo con il costo (es. F1=20k, F2=40k, F3=20k), possiamo usare le DTMC per il Risk Management.

Analisi di Sensibilità (Esempio C):

Se vogliamo che la probabilità di finire il progetto in 8 mesi sia almeno del 50% ($P \geq 0.5$), e definiamo p come la probabilità di successo della Fase 1:

$$p \times 0.8 \times 0.7 \geq 0.5 \implies p \geq 0.560.5 \approx 0.893$$

Risultato: La Fase 1 deve avere un'efficienza (probabilità di avanzamento) di almeno l'89.3%.

3. Reti di DTMC (Composizione di Sistemi)

I sistemi complessi sono spesso composti da sottosistemi interagenti. In una rete di DTMC, l'output di una catena diventa l'input di un'altra.

Il Modello di Composizione $K||P$

Consideriamo due catene, K e P , interconnesse:

- **Catena K :** Riceve input esterno v e l'output w di P . Produce output $y1$.
- **Catena P :** Riceve come input l'output $y1$ di K . Produce output z (finale) e w (feedback per K).

Definizione della Catena Risultante M

La composizione genera una nuova DTMC $M=(V, X1 \times X2, Z, p, g)$ dove:

1. **Lo Stato:** È il prodotto cartesiano degli stati delle due catene $(x1, x2)$.

2. **Funzione di Probabilità di Transizione Composta:**

La probabilità di passare alla nuova coppia di stati $(x1', x2')$ è il prodotto delle probabilità individuali, tenendo conto delle dipendenze:

$$p((x1', x2') | (x1, x2), v) = p1(x1' | x1, (g2, W(x2), v)) \times p2(x2' | x2, g1(x1))$$

- $p1$ dipende dal suo stato attuale $x1$, dall'input esterno v e dalla componente W dell'output di P .

- p_2 dipende dal suo stato attuale x_2 e dall'output di K .

3. Funzione di Output Composta:

L'output finale della rete è la componente Z dell'output della catena P :

$$g(x_1, x_2) = g_2, Z(x_2)$$